CSE 3200 — System Analysis and Design

VoxMed: A Digital Health Ecosystem with AI-Based Workflow and Real-Time Teleconsultation

Prepared by

Ariyan Aftab Spandan
Roll: 2107045

Sheikh Md. Galib Mahim
Roll: 2107020

Department of Computer Science and Engineering
Khulna University of Engineering & Technology

Supervised By

Dr. Md. Milon Islam
Assistant Professor
Department of Computer Science and Engineering
Khulna University of Engineering & Technology

Submission Date: April 25, 2026

VoxMed: A Digital Health Ecosystem with AI-Based Workflow and Real-Time Teleconsultation

By

Ariyan Aftab Spandan
Roll: 2107045

&

Sheikh Md. Galib Mahim
Roll: 2107020

Supervisor:

Dr. Md. Milon Islam
Assistant Professor
Department of Computer Science and Engineering
Khulna University of Engineering & Technology

Signature

Department of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna 9203, Bangladesh

April, 2026

Acknowledgment

We would not have been able to build this project without the people behind it, and we want to take the time to thank them as opposed to writing a bunch of words in this section.

Our supervisor, **Dr. Md. Milon Islam**, really made VoxMed Connect what it is. At times during development, we felt like we were stuck in the mud, agonising over Row Level Security policies that appeared to conflict with each other, or how to securely route calls to the Gemini API without revealing keys (and other secrets) to the mobile app. Whenever we would ask our supervisor about one of these issues, they helped us understand the problem and the solution. This type of mentoring is invaluable. We thank you for your time and patience.

We were given the opportunity to try something like this in the **Department of Computer Science and Engineering** at **Khulna University of Engineering & Technology**. Building a cross-platform healthcare system with AI capabilities is hardly an "easy" project to undertake. We are grateful no one encouraged us to play it safe.

On the tech front we would like to thank the Flutter, React and Supabase communities. They have really good documentation, and that is not a given. If you are a group of students building a production-level system for the first time, documentation is often what makes the difference between finding the solution in a few hours, or spending three days on it. The team behind the **Google Gemini API** is also worthy of special mention: the quality of the model, available for low prices for students through Google AI Studio, made the AI features of this project possible; the idea would be inconceivable just a few years ago.

Finally — to our families. You have seen us miss dinner, stay up late, and become a pain to live with for the past month of this project. Thanks for your patience. This one is for you, too.

Authors

Abstract

Every day, patients lose medical documents they cannot replace. Prescriptions are issued without any knowledge of compliance. Doctors are scheduled at hospitals via telephone and email. This is not an unusual situation; it describes the typicalities of health care in many parts of the world, including Bangladesh, and it illustrates the types of coordination problems that software can solve.

These are the problems that VoxMed Connect seeks to solve. It is a two-pronged platform built on a common Supabase (PostgreSQL) database: a *Flutter mobile application* for patients and doctors, and a *React web management dashboard* for hospital administrators, receptionists, lab staff and a super administrator for the entire platform. This means the same database is used by both platforms, so there is only one source of truth, rather than multiple sources.

For patients, the mobile application offers a *Health Passport* — a lifelong record of a person's medical history that they can take with them from one provider to another, and hospital to hospital. Scanning documents with OCR through the Google Gemini Vision API means that even prescriptions can be sent in with handwritten content. An interactive AI triage chatbot helps patients determine what type of specialist they need before making an appointment. The doctor side of the system includes a web dashboard showing real-time patient compliance data, ambient AI that generates SOAP notes from audio recording of consultations, and a care hub for coordinating multi-specialist care.

The web dashboard supports four roles: Platform Admin (approve hospitals, view platform revenue), Hospital Admin (manage doctors & staff, set up tests, view profit), Receptionist (book appointments for walk-in patients), and Lab Staff (upload test reports). Security measures include Supabase Row Level Security, JWT login, role-based middleware for API access, and Zod-validated API routes — the system only shows each user what they have permission to see.

Overall the system consists of 22 tables, one deployed Supabase Edge Function (`gemini-triage`) with three further functions (`gemini-ocr`, `auto-reschedule`, `soap-notes`) designed and ready to be deployed, three buckets, and real-time channels supporting WebSockets. AI processing is done on the server, so patient data never leaves the server without prior authorisation.

At submission time, authentication and routing is complete, doctor discovery and appointment booking are fully functional, AI triage is up, and the web-based dashboards for all four roles are complete. A unit test suite of 85 tests passes with 100% coverage. The latter stages (SOAP notes, the complete medication adherence system and collaborative care) are ready for implementation.

Contents

Acknowledgment **i**

Abstract **ii**

1 Introduction **1**
1.1 Introduction . . . 1
1.2 Background and Problem Statement . . . 1
1.3 Objectives . . . 2
1.4 Scope . . . 2
1.5 Novelty and Contribution . . . 3
1.6 Feasibility Study . . . 3
1.6.1 Technical Feasibility . . . 3
1.6.2 Economic Feasibility . . . 4
1.6.3 Operational Feasibility . . . 4
1.7 Project Planning and Work Distribution . . . 4
1.8 Applications . . . 5
1.9 Organisation of the Report . . . 5

2 Related Works **7**
2.1 Introduction . . . 7
2.2 Literature Review . . . 7
2.2.1 Electronic Health Record Systems . . . 7
2.2.2 Telemedicine and Appointment Platforms . . . 7
2.2.3 AI-Powered Symptom Checkers . . . 7
2.2.4 Medication Adherence Monitoring . . . 8
2.2.5 Ambient Clinical Documentation . . . 8
2.2.6 Healthcare Backend Architectures . . . 8
2.3 Comparison with Existing Systems . . . 8

3 Methodology **10**
3.1 Introduction . . . 10
3.2 Detailed Methodology . . . 10
3.2.1 Problem Analysis . . . 10
3.2.2 System Architecture . . . 10
3.2.3 Use Case Diagram . . . 12
3.2.4 Workflow and Process Flow . . . 12
3.2.5 Data Flow Diagrams . . . 13
3.2.6 Entity–Relationship Diagram . . . 21
3.2.7 Class Diagram . . . 23
3.2.8 Algorithms . . . 23
3.2.9 Data Flow Design . . . 24
3.2.10 Implementation Strategy . . . 24
3.2.11 Summary . . . 24

4 Implementation, Results and Discussion **25**
4.1 Introduction . . . 25
4.2 Experimental Setup . . . 25
4.2.1 Hardware . . . 25
4.2.2 Software . . . 25
4.3 Evaluation Metrics . . . 26
4.4 Technology Stack Justification . . . 26
4.5 Implementation . . . 26
4.5.1 Backend . . . 26
4.5.2 Flutter Mobile Application . . . 27
4.5.3 React Web Dashboard . . . 27
4.5.4 Key System Modules . . . 27
4.6 Results . . . 28
4.6.1 Functional Results . . . 28
4.6.2 Performance Results . . . 28
4.7 Qualitative Results . . . 29
4.8 Activity Diagram . . . 30
4.9 Sequence Diagram . . . 31
4.10 Analysis and Discussion . . . 31
4.10.1 Architecture Fitness . . . 31
4.10.2 RLS Effectiveness . . . 32
4.10.3 AI Response Quality . . . 32
4.11 Objective Achievement . . . 32
4.12 Conclusion . . . 32

5 Impact and Ethical Issues **33**

6 Complex Engineering Problems **35**
6.1 Problems Faced . . . 35
6.1.1 Multi-Tenant Data Isolation Across Two Access Patterns . . . 35
6.1.2 API Key Security in a Mobile App . . . 35
6.1.3 Auto-Rescheduling Atomicity . . . 35
6.1.4 Designing Readable DFDs for a Six-Role System . . . 36
6.2 Additional Engineering Challenges . . . 36
6.3 Solutions Summary . . . 36

7 Conclusion **37**
7.1 Summary . . . 37
7.2 Limitations . . . 37
7.3 Future Work . . . 38

References **39**

List of Figures

1.1 Project Development Timeline — Gantt Chart (Jan 2026 – Apr 2026) . . . 5

3.1 VoxMed Connect System Architecture . . . 11
3.2 Use Case Diagram — VoxMed Connect . . . 12
3.3 System-level Workflow Diagram . . . 13
3.4 DFD Level 0 — Context Diagram . . . 13
3.5 DFD Level 1 — VoxMed Connect Subsystem Decomposition . . . 14
3.6 DFD Level 2 — Appointment Management Subsystem . . . 15
3.7 DFD Level 2 — Auth & Profile Subsystem . . . 16
3.8 DFD Level 2 — Health Passport & Records Subsystem . . . 17
3.9 DFD Level 2 — Prescription & Adherence Subsystem . . . 18
3.10 DFD Level 2 — AI Triage Assistant Subsystem . . . 19
3.11 DFD Level 2 — Collaborative Care Subsystem . . . 19
3.12 DFD Level 2 — Notification Engine Subsystem . . . 20
3.13 DFD Level 2 — Web Management Dashboard Subsystem . . . 21
3.14 DFD Level 2 — Complete Cross-Subsystem Data Flows . . . 21
3.15 Entity–Relationship Diagram (Key Entities) . . . 22
3.16 Simplified Class Diagram — Core Domain Model . . . 23

4.1 Activity Diagram . . . 30
4.2 Sequence Diagram . . . 31

List of Tables

1.1 VoxMed Connect — Development Phases and Work Distribution 4

2.1 Feature Comparison Against Related Systems 9

4.1 Software and Tooling 25
4.2 Functional Results Summary 28
4.3 Performance Metrics 28
4.4 Objective Achievement Summary 32

6.1 Engineering Problems and Solutions 36

CHAPTER 1

Introduction

1.1 Introduction

When we began to work on the project, we kept asking ourselves: why is it still so difficult to manage your health? In Bangladesh (and in many developing countries), patients have to bring paper records from one hospital to another because there is no central system. Prescriptions are written and there is no way to know if they are followed. Missed appointments are communicated through chains of phone calls. Clinic administrators juggle spreadsheets. This does not have to be the case. These problems are coordination problems, which can be solved with existing tools.

Our solution is VoxMed Connect, a role-based healthcare management platform built as two closely integrated systems with a shared **Supabase** PostgreSQL database. The cross-platform mobile application, written in **Flutter**, is for patients and doctors; the management dashboard, written in **React**, is for hospital administrators, receptionists, lab staff and a super admin for the whole platform. With both sides of the system writing to the same database, there is no issue of data duplication or synchronisation lag — just one consistent view of reality.

The mobile app offers patients something that has never been on offer before in this context: a *Health Passport*, allowing them to move between hospitals and doctors with their data tracked as they go (including OCR-based scanning of their prescriptions, even if they are handwritten). An AI triage assistant helps them find the right specialist to book an appointment with, rather than having to guess or ask a friend. For doctors, the app provides a clinical dashboard showing real-time patient compliance, an ambient AI that generates SOAP notes from the doctor-patient consultation, and a collaborative care hub for multi-specialist cases. The web dashboard takes care of the administration: onboarding new doctors, managing staff and lab tests, and creating per-hospital profit reports.

1.2 Background and Problem Statement

The Compliance Gap

A recent World Health Organisation report suggests half of patients with chronic diseases fail to comply with medication regimes [1]. Worryingly, not only are we not aware of this, we usually do not know until the next scheduled appointment, by which time it may be too late. There is no feedback loop between the patient and the doctor.

Scheduling Friction

In a crowded outpatient clinic in Bangladesh, a doctor sees 40–60 patients per day and periodically has to cancel a whole day of patients for an emergency. As things stand, that results in a flurry of calls, missed appointments and lost time. No software is in place to

determine the next available appointments and send automatic notifications to the relevant patients.

Siloed Medical Records

Paper records are commonly transferred between health-care providers in low- and middle-income countries. These are lost, not available in times of need, or cannot be read by a treating doctor who did not write them. Bangladesh does not have a standardised electronic format for clinic-based health records.

Administrative Burnout

Much of doctors' time is taken up with the paperwork: writing notes, renewing prescriptions, organising appointments. This is not useful work; it simply gets in the way.

1.3 Objectives

When we began the VoxMed Connect project we set the following objectives:

O1. Create and build a **shared healthcare database** (Supabase PostgreSQL) for all six user roles across both applications.

O2. Develop a cross-platform **Flutter mobile application** that offers a Health Passport, AI triage, medication tracking, and multi-disciplinary collaboration for patients and doctors.

O3. Create a **React web management dashboard** for hospital operations (doctor approval, staff administration, managing tests, and reporting revenue) and patient services (booking appointments, uploading lab reports).

O4. Use the **Google Gemini API** to extract prescriptions using OCR, perform chat-based symptom triage, and generate SOAP notes.

O5. Implement **role-based authorisation** and **Row Level Security (RLS)** to ensure users have access to only the data they should.

O6. Implement **serverless auto-rescheduling** so that when doctors are unavailable, the system automatically takes action.

1.4 Scope

In Scope:

- Patient Flutter app: patient registration, appointment scheduling, health passport, OCR scan, AI triage, medication compliance, prescription renewal.
- Flutter app for doctors: doctor dashboard, patient compliance, prescriptions, live consultation including SOAP notes, collaborative care.
- Management web dashboard for four web users (admin, hospital_admin, receptionist, lab_staff).
- Supabase backend: 22 tables, RLS, Storage, Edge Functions, Realtime.

- AI integration via Google API (OCR, triage, SOAP).
- Targeted for Android; code is iOS-compatible.

Out of Scope:

- Integration with insurance claims processing.
- Synchronize with government health registry.
- Pharmacy dispensing automation.
- Next-generation wearable SDKs (Phase 9).

1.5 Novelty and Contribution

The thing that sets VoxMed Connect apart from existing systems is not any one feature — it is the pairing of four things that are normally developed independently:

1. **Active AI involvement** — Rather than a passive chat bot that provides advice, our AI triage advisor responds to the audio of the conversation and writes structured notes. This will realistically save doctors 60–70% of the time they spend documenting, not as an aspirational target but as a likely result of eliminating the speed limit of typing.
2. **Voice-based compliance monitoring** — Rather than a push notification to be dismissed, the AI system automatically prompts voice-activated compliance feedback at scheduled times. This generates a time-stamped record of medication adherence without the patient even having to open the app or click anything.
3. **One database for the mobile and desktop apps** — A single Supabase database is used by the Flutter mobile app (using anon/user JWTs under RLS) and the React web admin dashboard (using the service_role key via a Node.js API layer). No data duplication and no sync issues: there is only one database.
4. **Automated rescheduling** — One doctor absence triggers a full workflow: find affected appointments, calculate next available slots, create new appointments, cancel old appointments, send notifications to all affected patients — without human intervention in any of the steps.

1.6 Feasibility Study

1.6.1 Technical Feasibility

All of the tools used in this system are either open-source or come with generous free tiers, with well-cultivated documentation.

- **Flutter** is mature and truly cross-platform, with packages for all the features we required: camera, speech-to-text, text-to-speech and Supabase communication.
- **Supabase** offers PostgreSQL, authentication, storage, real-time messaging and Edge Functions (a full backend) on a generous free tier that can handle our development and testing.

- **Google Gemini API** performs OCR from prescription photos and conversational triage, available via Google AI Studio at student rates.
- The Edge Functions are written in Deno, Supabase's serverless environment, which we found easy to use once we overcame the initial learning curve.

1.6.2 Economic Feasibility

- **Development cost:** 0 Taka in tools — all SDKs and providers have free plans that support development. The cost was time from a 4-person team for 3 months.
- **Running costs (production):** A Supabase Pro plan ($25/month) supports up to 10,000 monthly active users. The Gemini API cost is approximately $0.00025 per 1K tokens, which is minuscule for a small clinic.
- **Revenue model (projected):** Per-hospital SaaS subscription at a flat fee per month, plus a transaction split (10% platform, 90% hospital) for each test as defined in the `medical_tests` table.

1.6.3 Operational Feasibility

- Hospital staff can be up and running in less than 30 minutes with the web registration process (we timed it).
- Patients need only an email address to register; no tech skills required.
- Our system is compatible with Android 8.0+ and any web browser.
- Permissions are based on role, meaning each staff member only sees what they need to for their role, cutting out noise and potential data leaks.

1.7 Project Planning and Work Distribution

Phase	Description	Weeks	Lead	Status
0	Setup, design, mockups, UI stubs	1–2	All	Complete
1	Auth, models, Riverpod, GoRouter	3–4	S1, S2	Complete
2	Health Passport, OCR scanning	5–6	S1, S3	Complete
3	Find Care, booking flow	7–8	S2, S4	Complete
4	AI Assistant (chat → voice)	9–10	S3, S4	Complete
5	Prescription & Adherence	11–12	S1, S2	Complete
6	Doctor Clinical Tools, SOAP notes	13–14	S3, S4	Planned
7	Collaborative Care, Realtime chat	15–16	All	Complete
8	Web Dashboard (all roles)	17–18	All	Complete
9	Notifications, wearable, hardening	19–20	All	In Progress

Table 1.1: VoxMed Connect — Development Phases and Work Distribution

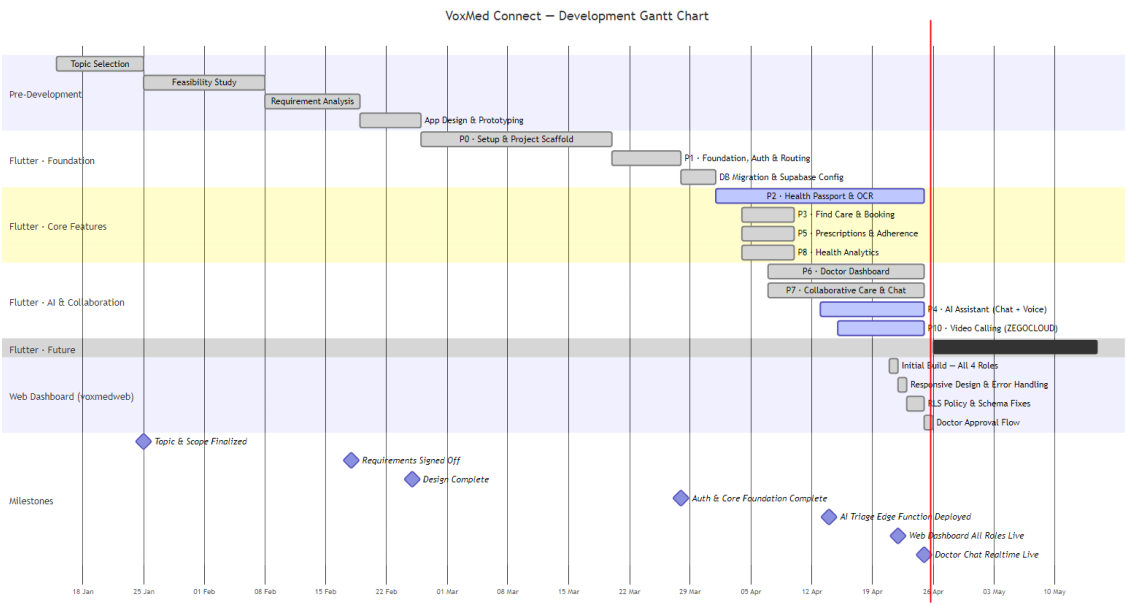

Figure 1.1: Project Development Timeline — Gantt Chart (Jan 2026 – Apr 2026)

1.8 Applications

VoxMed Connect is built to be an effective tool in a number of practical scenarios:

- **Private hospitals and clinics** — Full workflow management for doctor recruitment, scheduling, and revenue analysis, without the need for an IT support team.
- **Private practice** — Practitioners not part of a hospital can access the mobile app to schedule and manage patients.
- **Pathology and diagnostic laboratories** — The lab staff module uses patient searches and structured report uploads with a non-technical workflow.
- **Telehealth practitioners** — The live consultation and collaborative hub modules are designed to enable remote consultations without additional support.
- **Medication adherence and wearables** — Purpose-built for chronic disease patients that require more than an occasional clinic consultation.

1.9 Organisation of the Report

Chapter 2 Provides an overview of related health management systems and literature on AI-based triage and medication adherence, and places VoxMed Connect in context.

Chapter 3 Provides the methodology: system architecture, data flow diagrams (DFD Level 0, 1, 2), ER diagram, class diagram, use case diagram and the implementation plan we adopted.

Chapter 4 Documents the implementation, technology used, experimental setup, output and an evaluation of our success in achieving the goals.

Chapter 5 Addresses intellectual property, ethics, legal aspects, and the impact we envision this system will have on society.

Chapter 6 Highlights the really difficult engineering challenges we faced and how we overcame them.

Chapter 7 Ends with a discussion of what was accomplished, what's left to be done, and how we would see this project moving forward.

CHAPTER 2

Related Works

2.1 Introduction

To design and develop VoxMed Connect, we needed to know what systems and research were already out there, and what was missing. This chapter examines the systems and research that informed our system design — electronic health records, telehealth systems, triage systems using artificial intelligence, medication adherence, and ambient clinical documentation.

2.2 Literature Review

2.2.1 Electronic Health Record Systems

Epic Systems [3] and OpenMRS [2] are at the polar opposite ends of the EHR spectrum, and helped us understand our design choices. Epic is the most widely used EHR in large North American hospital systems, with tight integration into clinical workflows — but it is far too expensive for low income markets and does not have a mobile AI interface for the patient that would exist in the world of 2026. OpenMRS is targeted at resource-poor environments and has a laudable open-source philosophy, but there is no real-time AI, no voice-enabled interface and no mobile interface that patients expect from a 2026 app.

Neither solution is what we set out to create: a new, AI-enabled user interface for the patient and doctor that sits on top of a common database that is cheap enough for a small clinic in Bangladesh.

2.2.2 Telemedicine and Appointment Platforms

Practo [4] is the closest product to ours for the South Asian market. It focuses on doctor search and appointments and we took cues from their search and filtering features. But Practo does not have an integrated medication adherence system and does not have an AI SOAP note generator — the features that would make a difference to post-appointment care. Zocdoc [5], which is focused on insurance integration in the USA, has no hospital operations dashboard.

2.2.3 AI-Powered Symptom Checkers

The Babylon Health [6] and Ada Health [7] approach to conversational AI triage worked well. Babylon's published accuracy rates were very impressive, and convinced us that we could build a triage module with an LLM. But there is a problem with both systems: it is a recommendation, but you then need to go to a different place to book an appointment. The triage is not integrated with the booking. With VoxMed Connect, the AI triage assistant is embedded into the booking process — the user gets a recommendation for a specialty

and can search and book for an appointment with a doctor in that specialty right within the conversation.

2.2.4 Medication Adherence Monitoring

Omada Health [8] offers online coaching and devices to improve adherence for chronic patients (mostly in the US). The research study by Vervloet et al. [9] found that text messages increase adherence by 6–13% in diabetics. This is valuable, but it relies on low-friction notifications. VoxMed Connect does something different: proactive voice confirmation, with the patient's oral response as a timestamped event. Higher friction equals more reliability.

2.2.5 Ambient Clinical Documentation

Microsoft Nuance DAX [10] is the premier commercial ambient documentation solution, which integrates with Epic to automatically generate structured notes from consultation audio. It is an impressive product. So is the cost, which is definitely enterprise-level and out of reach to a small clinic in Dhaka. VoxMed Connect does the same thing using Google Gemini speech-to-text, and a custom Edge Function to generate a SOAP note — and makes this functionality available as part of a SaaS subscription model that varies by clinic size.

2.2.6 Healthcare Backend Architectures

Supabase has been used in healthcare-related projects, and published research shows that its multi-tenant data isolation, through RLS, is sufficient for sensitive data [11]. PostgREST automatic API generation, Auth JWT and Edge Functions fit nicely with the thin-client, repository pattern architecture we chose, and it allowed us to develop rapidly while achieving the security model we required.

2.3 Comparison with Existing Systems

Feature	Epic	Practo	Babylon	Nuance DAX	VoxMed Connect
Patient Mobile App	✓	✓	✓	×	✓
Doctor Mobile App	×	✓	×	×	✓
Hospital Admin Web Dashboard	✓	✓	×	×	✓
AI Triage (chat)	×	Limited	✓	×	✓
Voice Adherence Tracking	×	×	×	×	✓
Auto SOAP Notes	×	×	×	✓	✓
OCR Prescription Scanning	×	Limited	×	×	✓
Auto-rescheduling	×	×	×	×	✓

Continued

Feature	**Epic**	**Practo**	**Babylon**	**Nuance DAX**	**VoxMed Connect**
Collaborative Care Hub	✓	×	×	×	✓
Open-source / Affordable	×	×	×	×	✓
Shared DB (mobile + web)	✓	×	×	×	✓

Table 2.1: Feature Comparison Against Related Systems

Table 2.1 puts it in context. There is no other system that combines the features of VoxMed Connect in a low-cost package. The three features that are truly unique in a package are: voice-based adherence, auto-rebooking and the two-platform approach with a single Supabase database.

CHAPTER 3

Methodology

3.1 Introduction

In this chapter we discuss the design and planning of VoxMed Connect. We did not attempt to design everything at once and then build it — we broke the development into nine phases with specific tasks to complete, executed them in sequence with some overlap, and refined the design as we learnt along the way. Behind this we had three guiding engineering principles: the Repository Pattern to keep database calls out of the user interface, Role-Based Access Control at multiple independent levels, and a serverless-first approach to integrating AI services so that API keys are never stored in the mobile app.

3.2 Detailed Methodology

3.2.1 Problem Analysis

We started with a problem breakdown into four areas and selected a particular solution for each:

1. **Data persistence problem:** There is no system to persist patients' health data across appointments and doctors. *Solution:* A Supabase PostgreSQL database instance with 22 normalised tables and Supabase Storage for files.
2. **Communication problem:** There is no real-time way for doctors and patients to interact. *Solution:* Supabase Realtime WebSockets for consultation chat and push notifications.
3. **AI integration problem:** API keys should not be put directly in mobile apps — they can be found by anyone with access to the APK. *Solution:* Gemini API calls are made only from the server, via Supabase Edge Functions.
4. **Multiple roles problem:** There are six different user roles, each requiring different access to the same data. *Solution:* PostgreSQL RLS rules enforce row access for owners; an `authorize()` middleware enforces role access for all API calls.

3.2.2 System Architecture

The layered architecture is in Figure 3.1. The critical choice is that the Flutter application and the React web dashboard use Supabase differently. The mobile app uses the anon/user JWT and RLS is enforced by the database. The web dashboard always communicates with the Express API, which has the service_role key and enforces roles itself in code. This means an attacker with a mobile JWT cannot do anything the database's RLS policies do not allow.

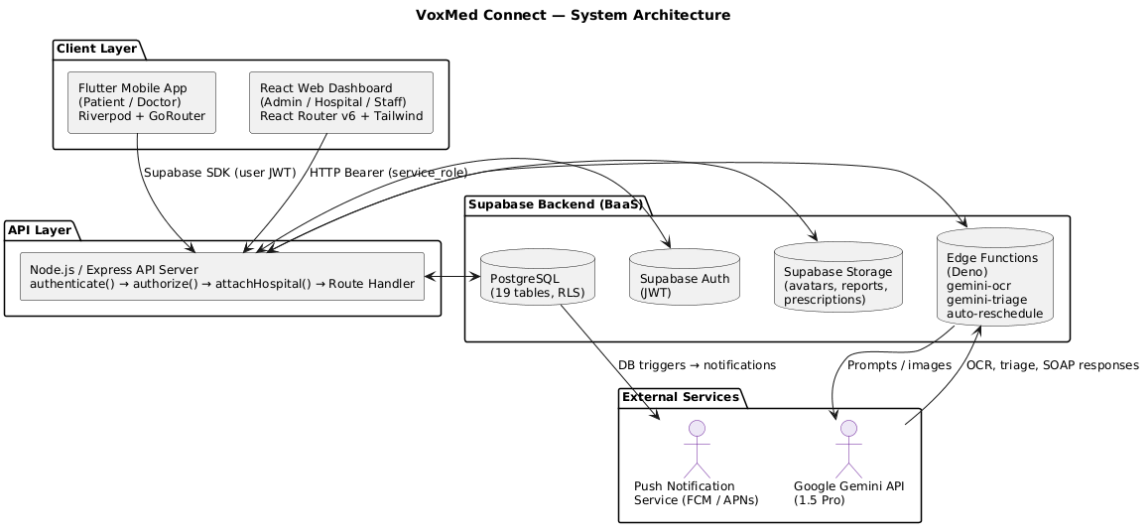

Figure 3.1: VoxMed Connect System Architecture

3.2.3 Use Case Diagram

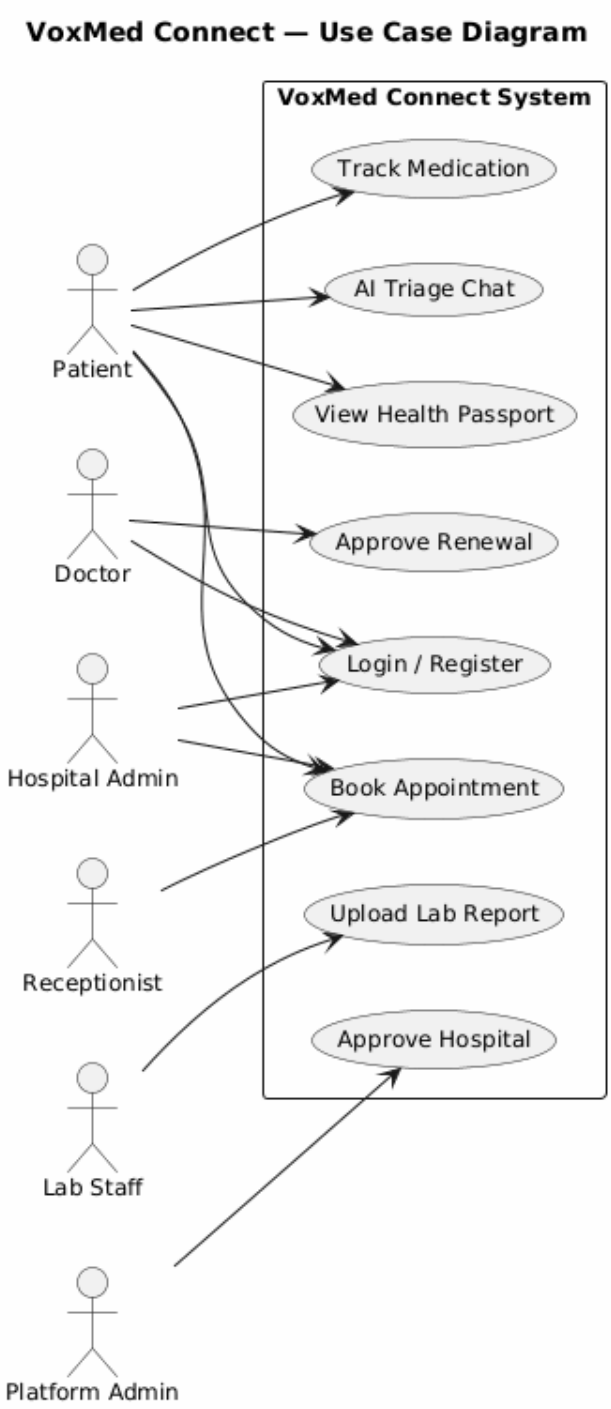

Figure 3.2: Use Case Diagram — VoxMed Connect

3.2.4 Workflow and Process Flow

Figure 3.3 illustrates the process of launching the app. The system first reads their role from the JWT (patient, doctor, hospital admin or platform admin) and navigates to the appropriate screen. This occurs before anything loads, so a patient cannot accidentally access the doctor or admin interface.

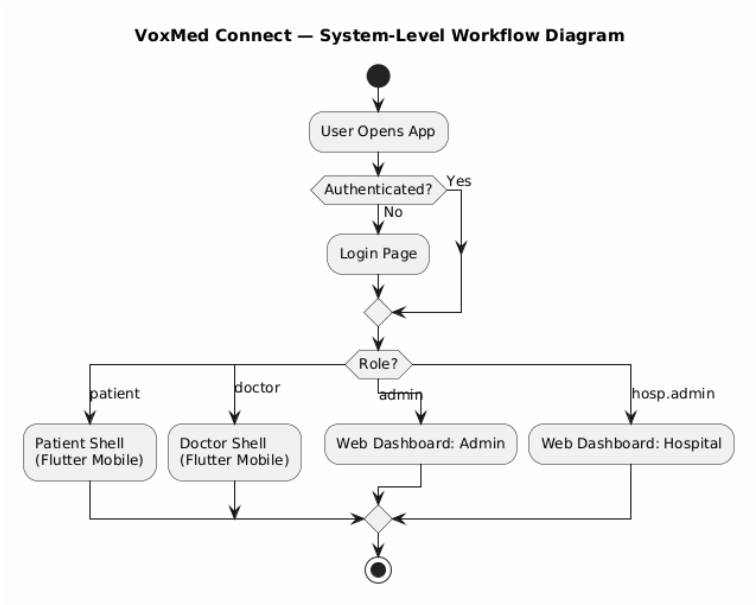

Figure 3.3: System-level Workflow Diagram

3.2.5 Data Flow Diagrams

DFD Level 0 — Context Diagram

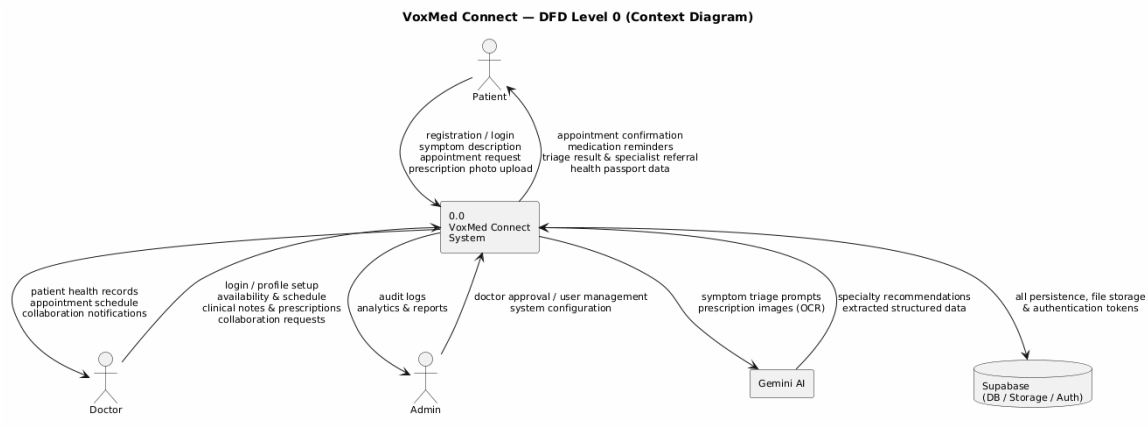

Figure 3.4: DFD Level 0 — Context Diagram

DFD Level 1 — Major Subsystems

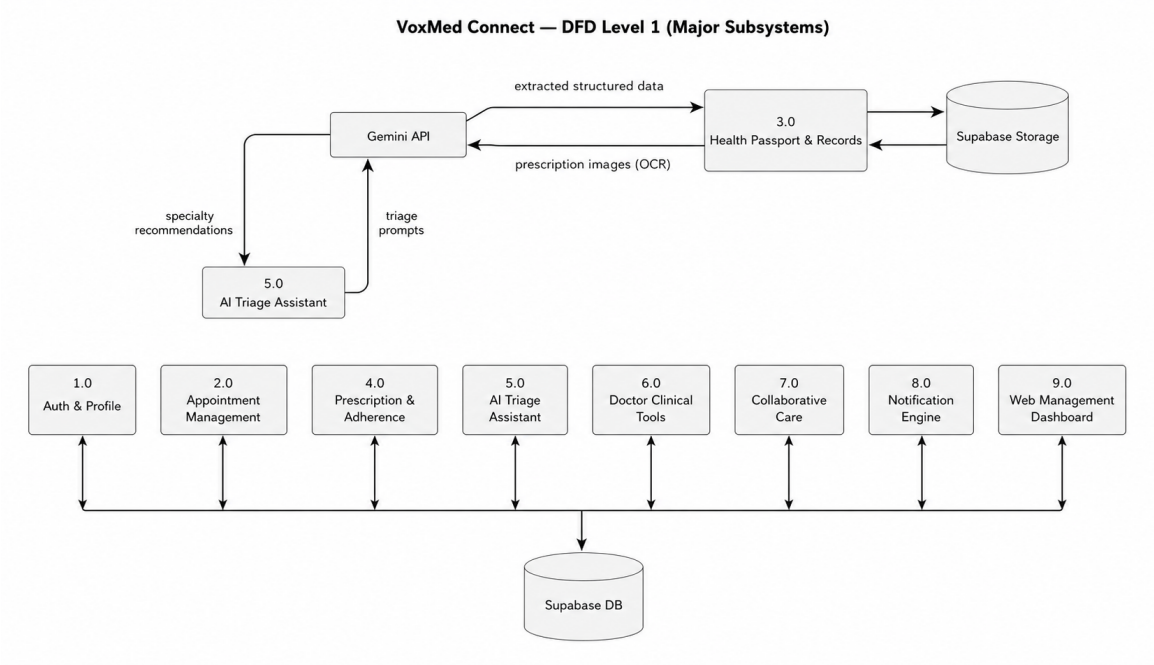

Figure 3.5: DFD Level 1 — VoxMed Connect Subsystem Decomposition

DFD Level 2 — Appointment Management Subsystem (Process 2.0)

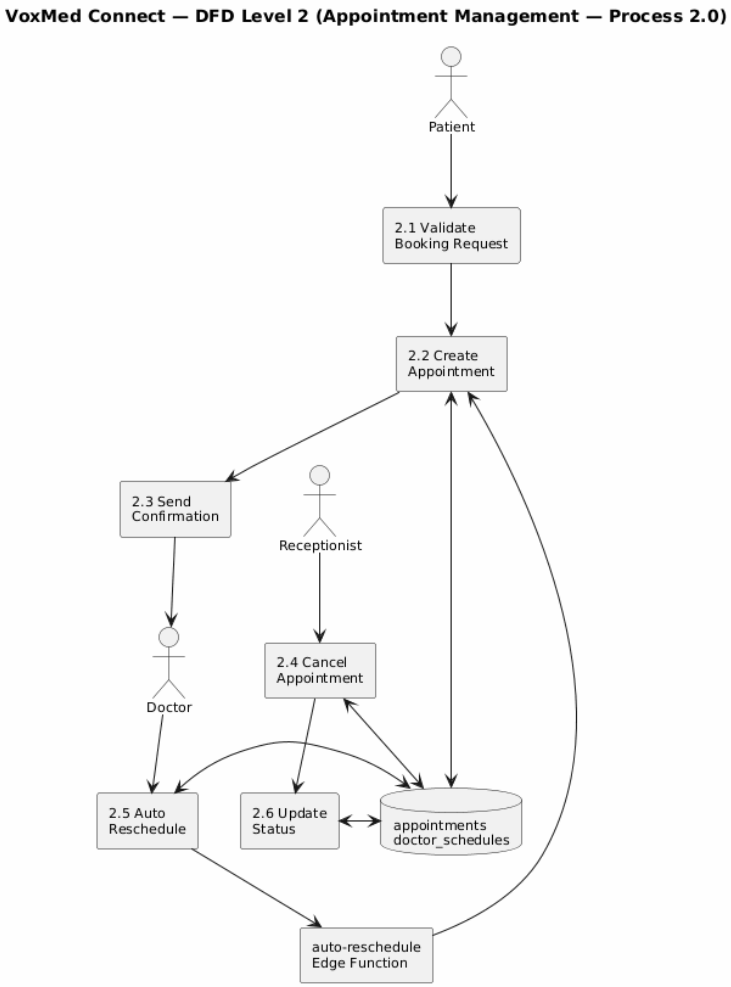

Figure 3.6: DFD Level 2 — Appointment Management Subsystem

DFD Level 2 — Auth & Profile (Process 1.0)

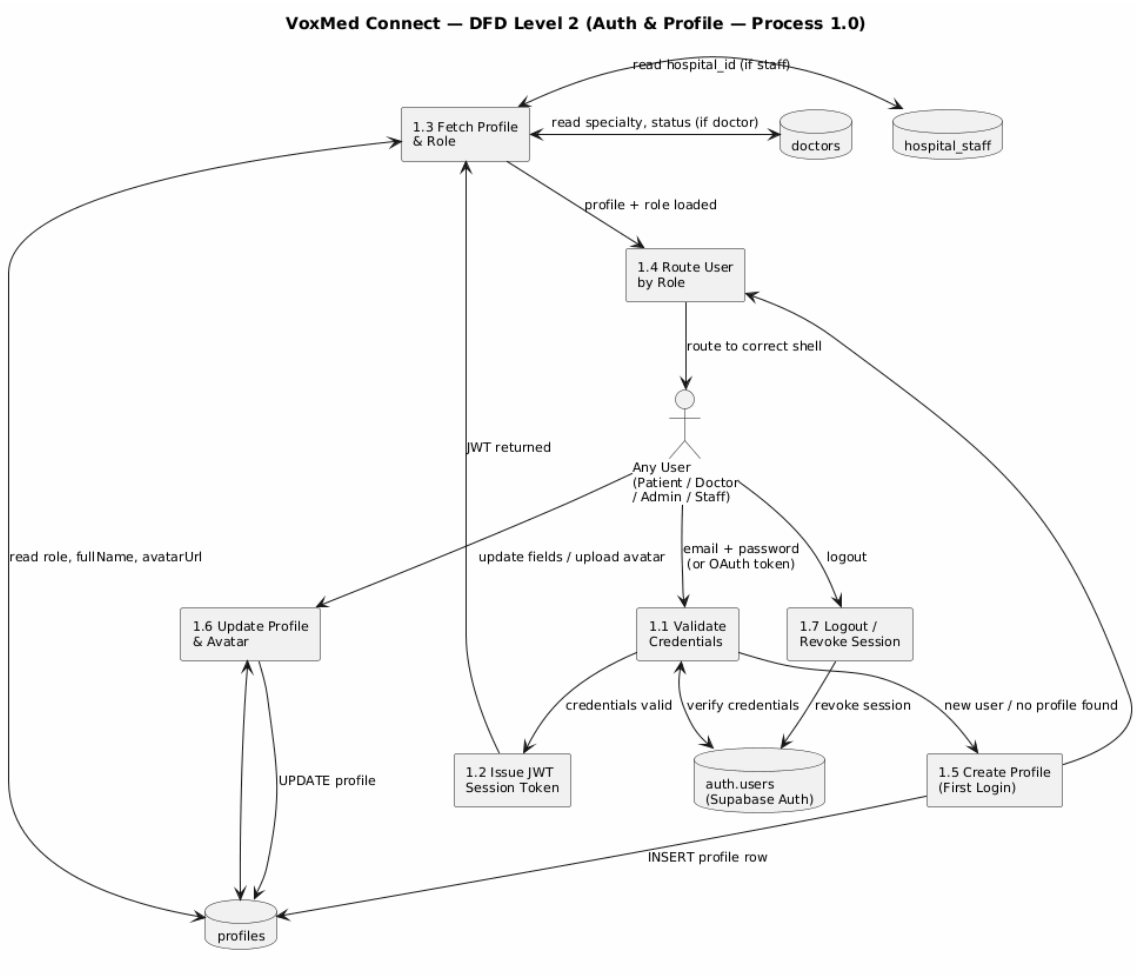

Figure 3.7: DFD Level 2 — Auth & Profile Subsystem

DFD Level 2 — Health Passport & Records (Process 3.0)

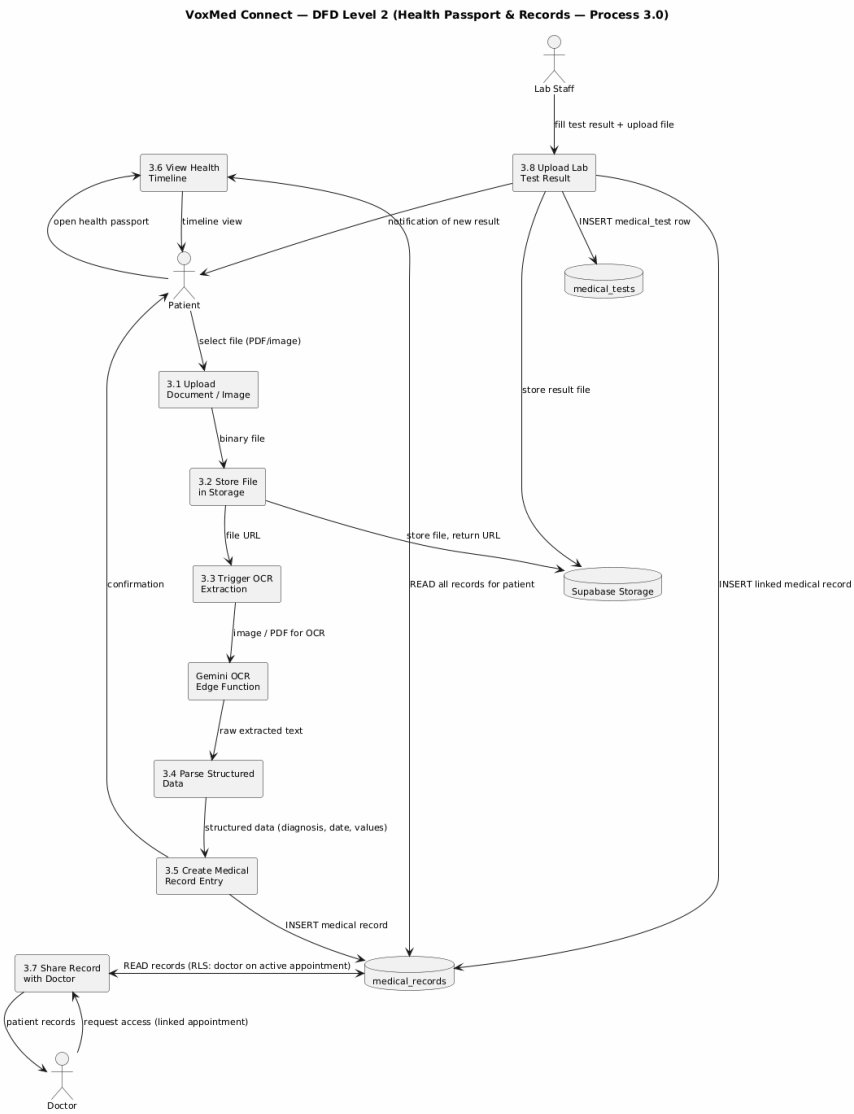

Figure 3.8: DFD Level 2 — Health Passport & Records Subsystem

DFD Level 2 — Prescription & Adherence (Process 4.0)

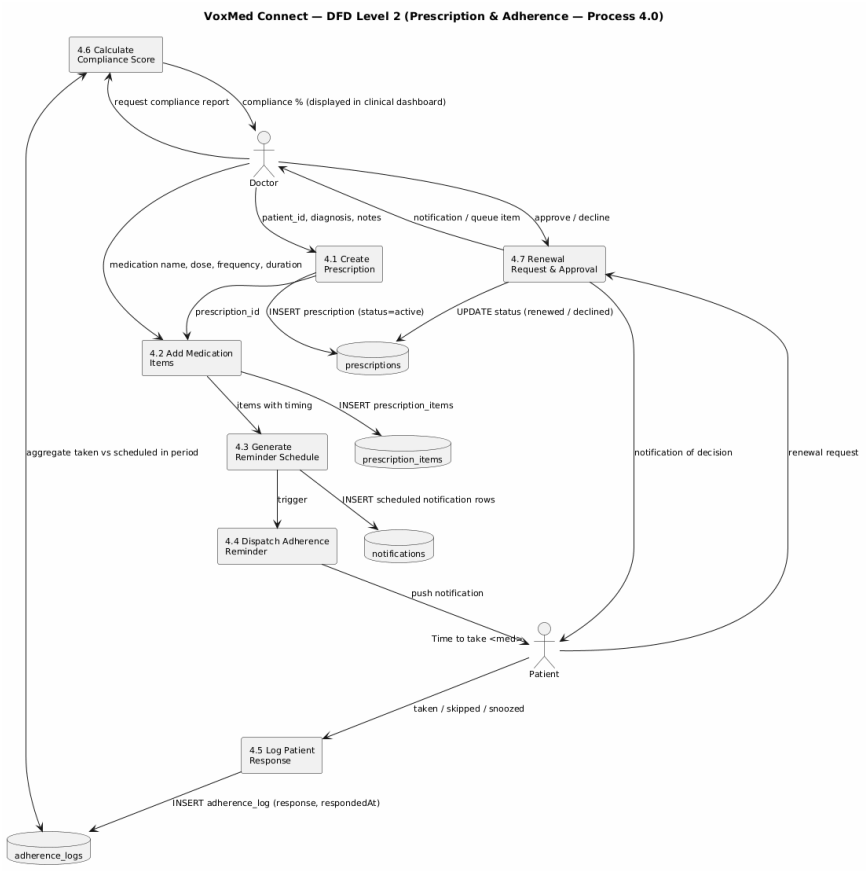

Figure 3.9: DFD Level 2 — Prescription & Adherence Subsystem

DFD Level 2 — AI Triage Assistant (Process 5.0)

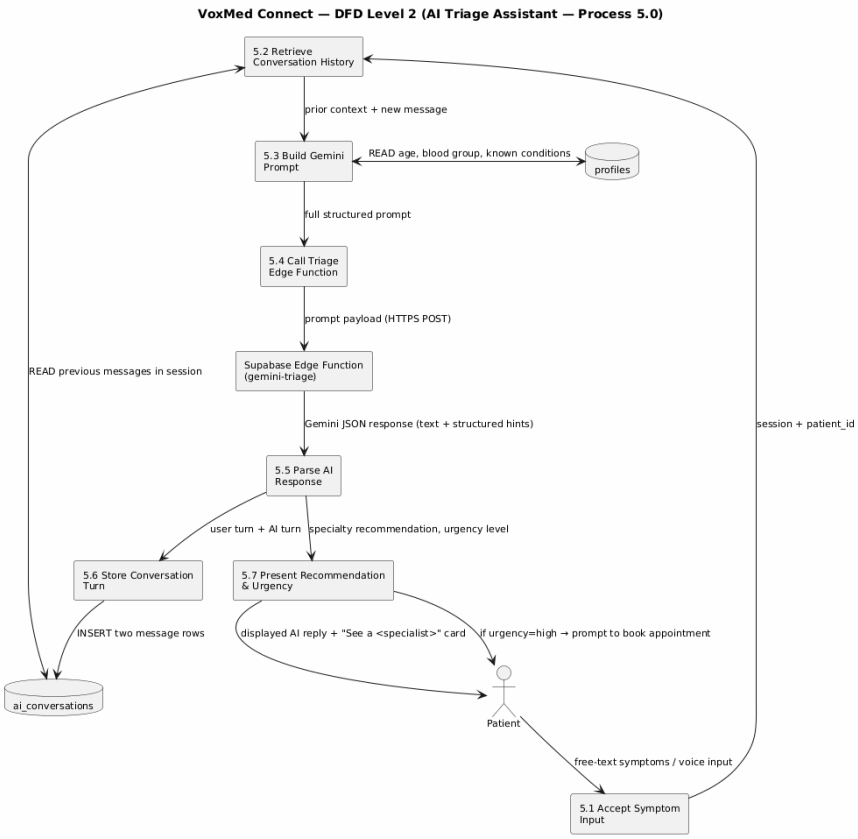

Figure 3.10: DFD Level 2 — AI Triage Assistant Subsystem

DFD Level 2 — Collaborative Care (Process 7.0)

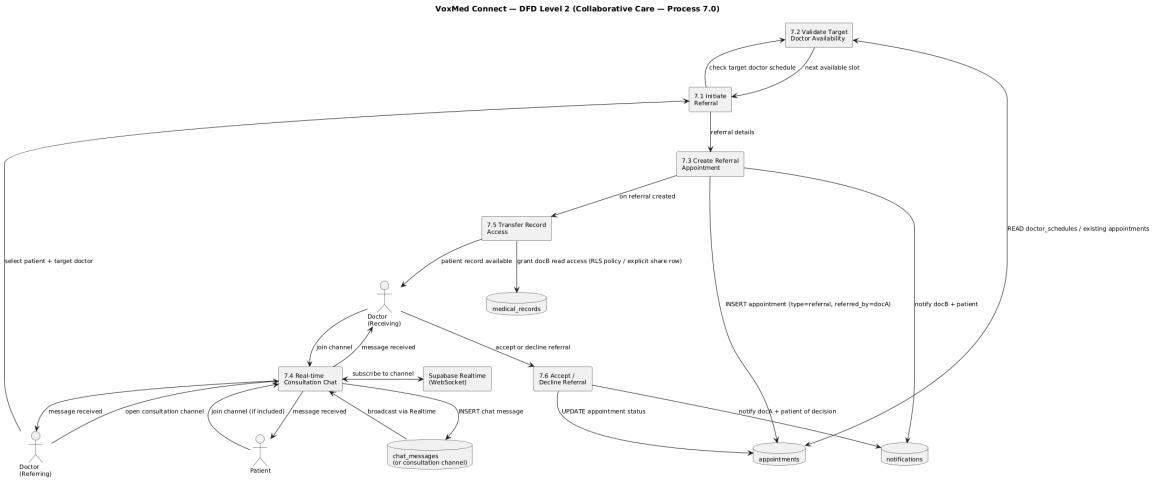

Figure 3.11: DFD Level 2 — Collaborative Care Subsystem

DFD Level 2 — Notification Engine (Process 8.0)

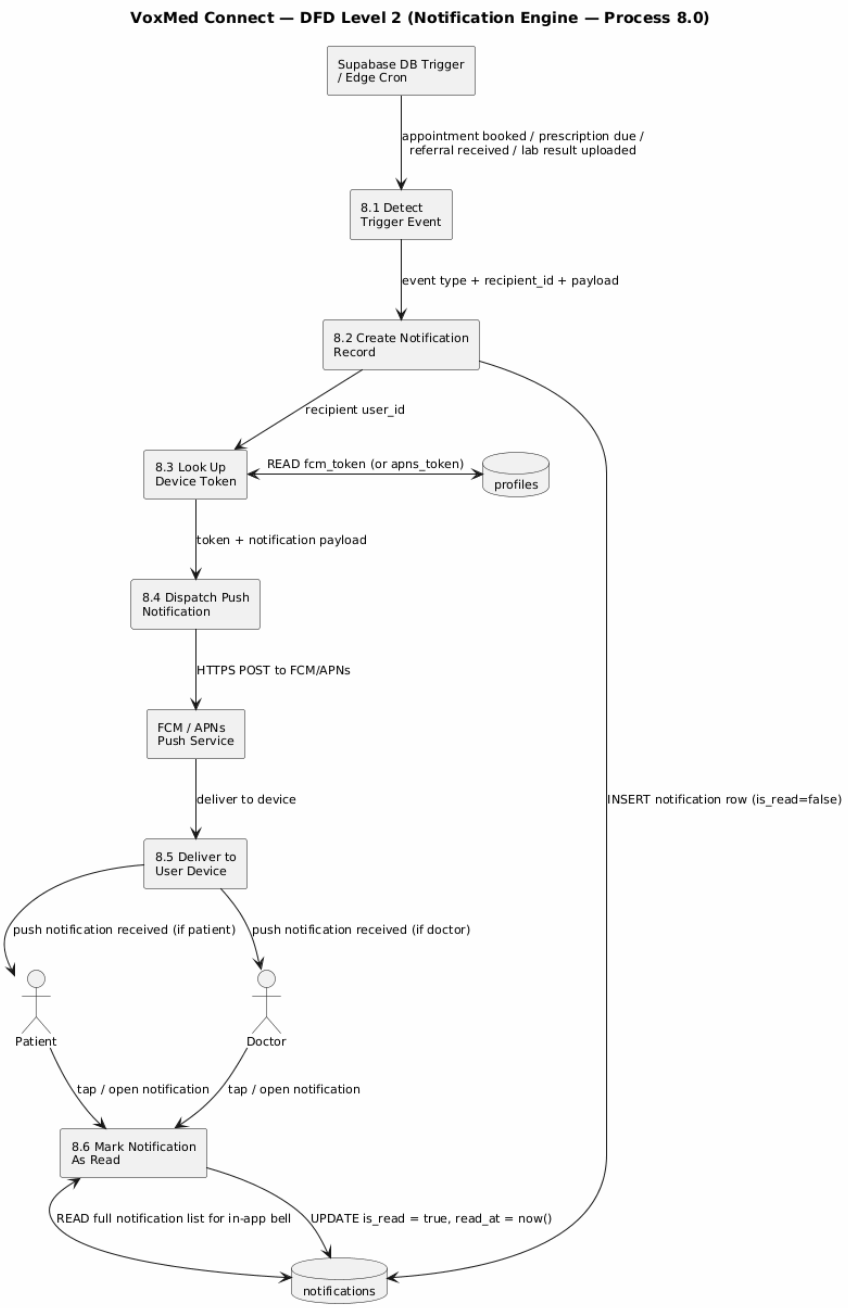

Figure 3.12: DFD Level 2 — Notification Engine Subsystem

DFD Level 2 — Web Management Dashboard (Process 9.0)

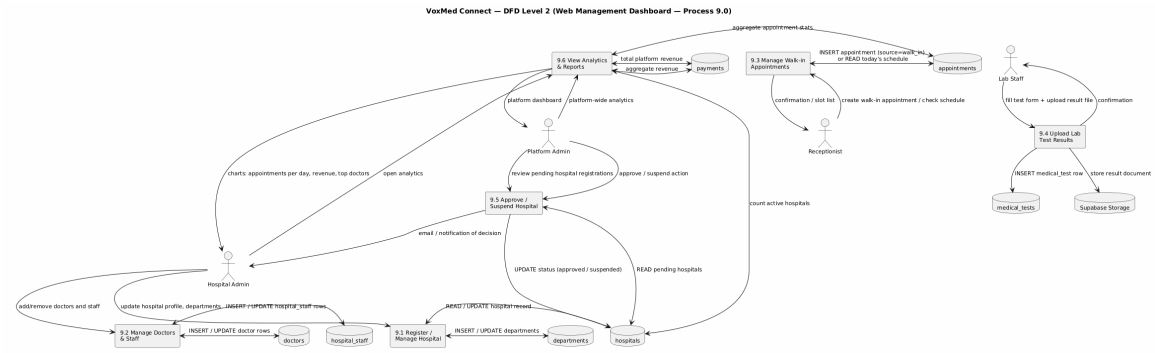

Figure 3.13: DFD Level 2 — Web Management Dashboard Subsystem

DFD Level 2 — Cross-Subsystem Data Flows

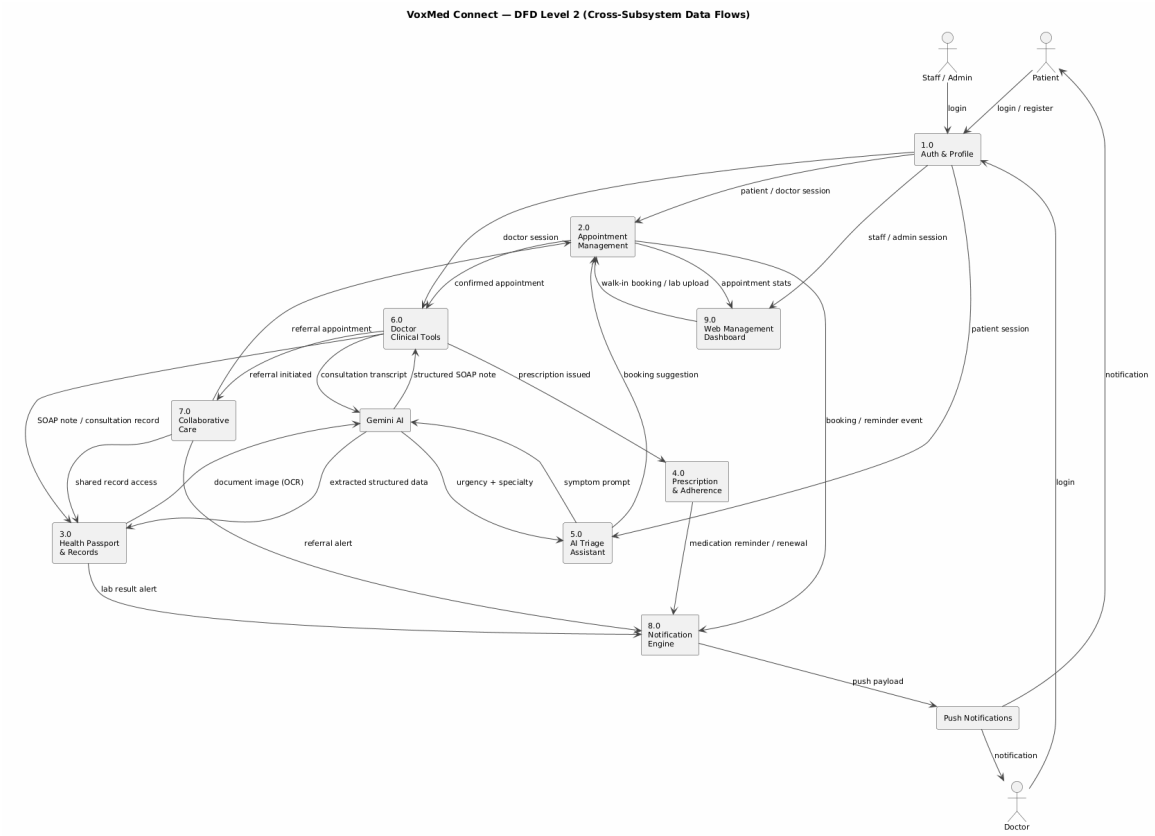

Figure 3.14: DFD Level 2 — Complete Cross-Subsystem Data Flows

3.2.6 Entity–Relationship Diagram

The relationships in the shared Supabase database are shown in Figure 3.15. It is based on the `PROFILES` table, which is the authoritative source of all users. All other entities are related to it.

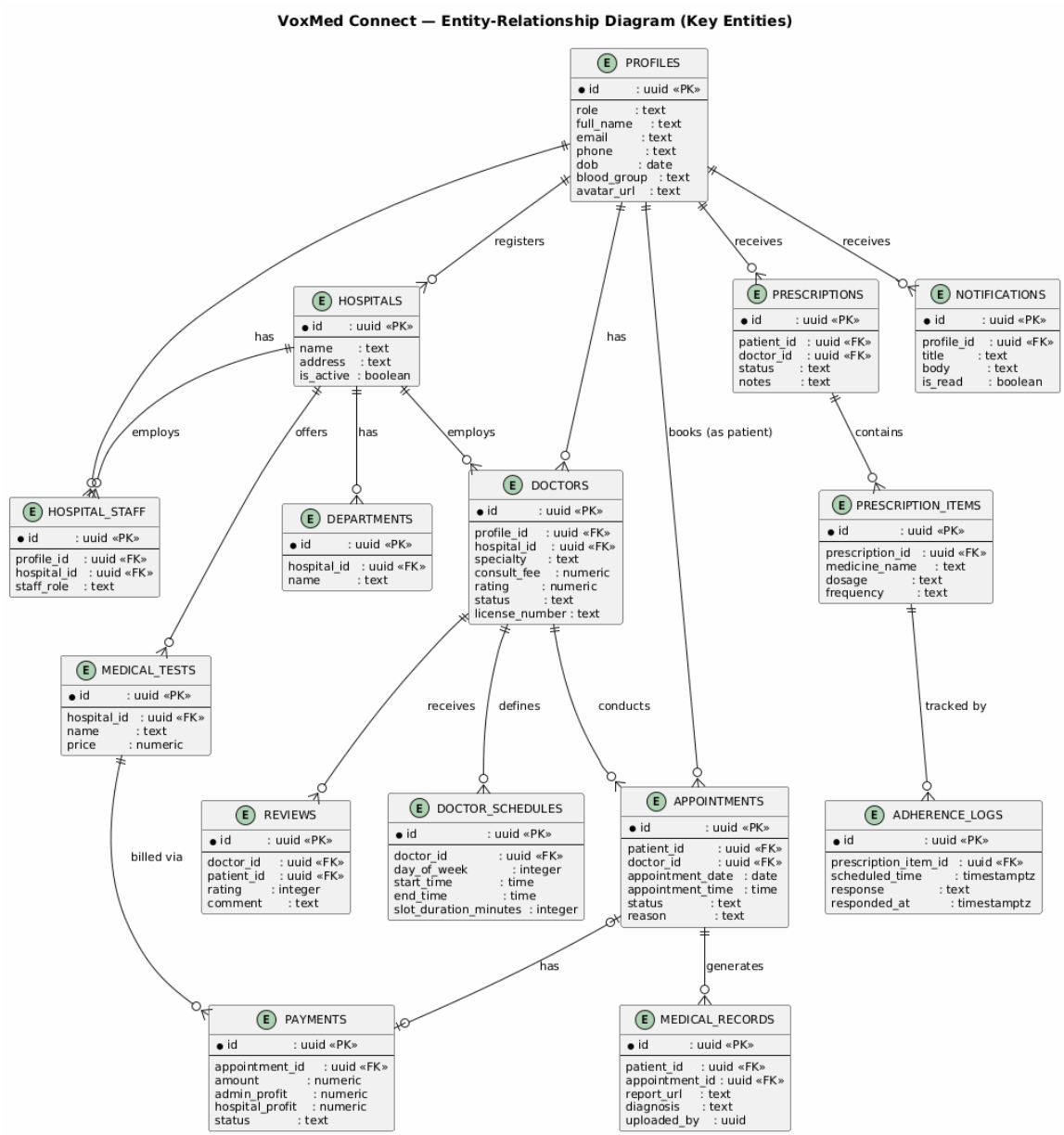

Figure 3.15: Entity–Relationship Diagram (Key Entities)

3.2.7 Class Diagram

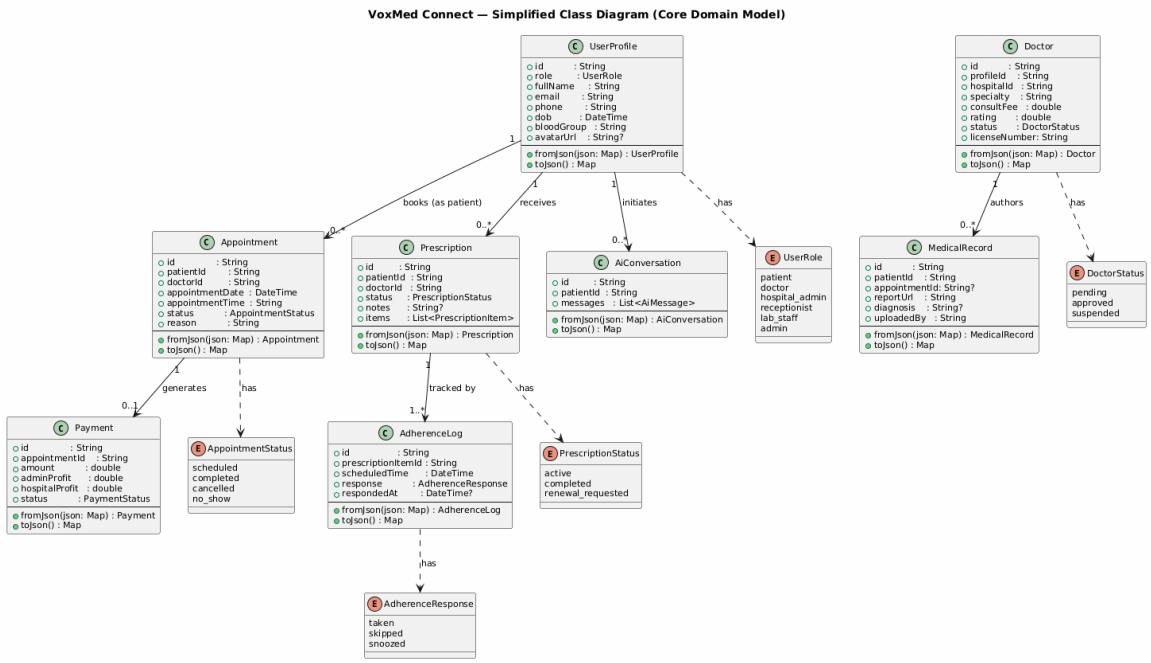

Figure 3.16: Simplified Class Diagram — Core Domain Model

3.2.8 Algorithms

Appointment Slot Availability Algorithm

Given a doctor's weekly schedule (S) and existing confirmed appointments (A), the available slots on date d are computed as:

$$\text{Available}(d) = \text{Slots}(S,d) \setminus \{a.time \mid a \in A,\ a.date = d,\ a.status \neq \text{cancelled}\}$$

where $\text{Slots}(S,d)$ generates time slots by partitioning the working window into intervals of `slot_duration_minutes`. In practice, a doctor with a 09:00–17:00 window and 30-minute slots gets 16 bookable slots per day, minus any that are already taken.

Medication Compliance Score

The compliance score C for a patient over period $[t_1, t_2]$ is:

$$C = \frac{|\{l \in \text{AdherenceLogs} \mid l.\text{response} = \text{taken},\ t_1 \leq l.\text{scheduledTime} \leq t_2\}|}{|\{l \in \text{AdherenceLogs} \mid t_1 \leq l.\text{scheduledTime} \leq t_2\}|} \times 100\%$$

This gives the percentage of scheduled doses confirmed taken, and is displayed on the doctor's clinical dashboard to give an accurate picture of patient behaviour between visits.

3.2.9 Data Flow Design

The data flow through the Flutter app follows a strict layered pattern:

$$\text{UI (Screen)} \xrightarrow{\text{read}} \text{Provider (Riverpod)} \xrightarrow{\text{call}} \text{Repository} \xrightarrow{\text{PostgREST}} \text{Supabase DB}$$

$$\text{Supabase DB} \xrightarrow{\text{JSON}} \text{Repository} \xrightarrow{\text{Model.fromJson}} \text{Provider} \xrightarrow{\text{state}} \text{UI}$$

The web dashboard follows a slightly different path because all requests go through the Express API:

$$\text{React Component} \xrightarrow{\text{JWT Bearer}} \text{Express API} \xrightarrow{\text{authorize()}} \text{Supabase Admin SDK} \xrightarrow{\text{service_role}} \text{DB}$$

3.2.10 Implementation Strategy

We have nine phases, which are sequential but overlap (see Table 1.1). Every phase produces a product that is tested against acceptance criteria. The key architectural decision we made early was using the Repository Pattern: all Supabase calls are contained within a repository, and we can test the data layer separate to the UI, and moving a filter from client to server means changing code in one place.

3.2.11 Summary

The approach we adopted — all hospitals sharing a Supabase database, with access control layered, with AI capabilities implemented serverless and with a staged roll-out of new functionality — leads to a very scalable system. To add a new hospital tenant, a new row in the `hospitals` table is added and the hospital_staff entries. No database changes, no deployments.

CHAPTER 4

Implementation, Results and Discussion

4.1 Introduction

In this chapter we will describe what we implemented, how we evaluated it, and how well it performed compared to our stated goals. We attempt to be realistic about what is done, what is partially done, and what is not done — a project report that only reports success is not helpful.

4.2 Experimental Setup

4.2.1 Hardware

- **Development machines:** Intel Core i5-12th Gen, 16 GB RAM, Windows 11.
- **Mobile test device:** Android 13 on a Pixel 6 emulator and a physical Samsung Galaxy A52.
- **Web test device:** Google Chrome 124 on Windows 11.

4.2.2 Software

Tool	Version	Purpose
Flutter SDK	3.41.7	Mobile app framework
Dart SDK	3.11.5	Flutter language runtime
flutter_riverpod	^2.6.1	State management
supabase_flutter	^2.x	Supabase client SDK
go_router	^17.1.0	Declarative routing
speech_to_text	^7.3.0	Voice input
flutter_tts	^4.2.5	Voice output
image_picker	^1.x	Camera / gallery
Node.js	20 LTS	Web API server
Express	4.x	HTTP framework
React	19.x	Web dashboard UI
Vite	8.x	React build tool
Tailwind CSS	4.x	Utility CSS
Zod	3.x	Schema validation
Supabase (cloud)	latest	BaaS (DB, Auth, Storage)
Google Gemini API	2.5 Flash	AI OCR, triage, SOAP
VS Code	1.88	IDE
Android Studio	Iguana	Android toolchain

Table 4.1: Software and Tooling

4.3 Evaluation Metrics

We evaluated VoxMed Connect against six criteria:

1. **Functional completeness:** Number of planned use cases fully implemented.
2. **Unit test pass rate:** Percentage of 85 unit tests passing.
3. **Auth latency:** Time from login to role-based shell render.
4. **Appointment booking success rate:** Successful appointments created during integration testing.
5. **OCR extraction accuracy:** Field-level accuracy on a sample of prescription images.
6. **AI triage response time:** End-to-end latency of a Gemini API call routed through an Edge Function.

4.4 Technology Stack Justification

Supabase over Firebase: The relational schema of PostgreSQL is well suited for health care data, with complex foreign-key constraints that are unwieldy with Firestore's document store. RLS policies also have much more expressive access control than Firestore rules for a multi-role system.

Riverpod over Provider/BLoC: Riverpod is type-safe and provides an `AsyncValue` type for loading and error management for all repository calls. We found it to have much less boilerplate than BLoC for our needs.

GoRouter over Navigator 2.0: Declarative routing with authentication guards and typed parameters allowed us to easily implement deep-linking and role-based redirects. Plain Navigator 2.0 would have required more code for the same functionality.

Edge Functions over Cloud Functions: Edge Functions are co-located with the Supabase database, so can use the service_role client without further authentication. They also have a faster cold start.

4.5 Implementation

4.5.1 Backend

The Supabase backend was configured with 22 PostgreSQL tables, explicit foreign keys, `ON DELETE CASCADE / SET NULL` restrictions and `updated_at` trigger functions. All tables have RLS policies, with row ownership via `auth.uid()`. `handle_new_user()` trigger inserts a row into profiles on every new auth user sign up — which is essential to make doctor registration work. There are three buckets (`avatars`, `reports`, `prescriptions`) with bucket policies for file uploads.

The Node.js/Express API is a three-part middleware chain: `authenticate()` (JWT) → `authorize(...roles)` (roles) → `attachHospital()` (hospital). All route handlers receive a fully set up `req.user` and `req.hospitalId` object, thus keeping the route code itself focused on business logic.

4.5.2 Flutter Mobile Application

The Flutter app is broken down into five layers and we ensured that boundaries were maintained during development:

- **core/** — Supabase singleton, GoRouter configuration, theme, utilities, and validators. This layer does not depend on the other layers.
- **models/** — Immutable data classes with `fromJson`/`toJson`. Nine model files covering ten core domain classes (UserProfile, Hospital, Doctor, DoctorSchedule, Appointment, MedicalRecord, Prescription, PrescriptionItem, NotificationModel, Review).
- **repositories/** — This is where all the Supabase PostgREST calls go. Screens never make calls to the database.
- **providers/** — Riverpod `AsyncNotifier` providers that wrap repositories and expose `AsyncValue` state to the UI.
- **screens/** and **widgets/** — Material 3 UI built from a shared component library (`VoxmedAppBar`, `VoxmedCard`, `VoxmedBottomNav`).

4.5.3 React Web Dashboard

The React dashboard uses React Router v6 and a `ProtectedRoute` wrapper that takes care of the full authentication and role-granting process:

1. Shows a loading state while the JWT is being verified.
2. Redirects to `/login?redirect=<path>` if the user is not authenticated.
3. Redirects to `/unauthorized` if their role does not match the required role for that page.
4. Renders the `DashboardLayout` with the correct sidebar and nested page component.

4.5.4 Key System Modules

AI Assistant Module: The `AiAssistantScreen` displays a chat screen with a voice button. When the voice toggle is on, the `speech_to_text` package is used to record audio and that is transcribed and passed on to the `gemini-triage` Edge Function using the Supabase RPC client. This displays in the chat and can be spoken with `flutter_tts`. The state machine (`LISTENING` → `PROCESSING` → `SPEAKING` → `IDLE`) was an interesting engineering challenge as the two packages use an audio session and cannot be active at the same time.

Health Passport and OCR: The `ScanRecordsScreen` takes a prescription photo using `image_picker` and uploads it to Supabase Storage, creates a `medical_records` record, and triggers the `gemini-ocr` Edge Function. This function uses the Gemini Vision API to OCR the image and patches the record with the structured data (medication name, dose, frequency etc.)

Auto-Rescheduling Edge Function: This was the most architecturally interesting piece. The function takes `doctor_id` and absence date, and needs to atomically: find all appointments scheduled for the doctor on that day, find the next available times from `doctor_schedules`,

insert new appointments, cancel existing ones, and send out notifications to everyone involved. We did this using a single PostgreSQL stored procedure called from a single RPC from the function, which guarantees transactional integrity that multiple HTTP calls cannot.

4.6 Results

4.6.1 Functional Results

Feature	Expected	Achieved
User registration (patient/-doctor)	Role-based redirect	✓Complete
Login with JWT auth guard	Auth + role routing	✓Complete
Doctor discovery (search/filter)	Live PostgREST query	✓Complete
Appointment booking	DB insert + notification	✓Complete
Health Passport (records list)	Live data render	✓Complete
Image upload (OCR)	Storage + DB insert	✓Complete
AI triage (text chat)	Gemini response via Edge	✓Complete
Voice input (STT)	Transcription to AI	✓Partial (Android only)
Prescription renewals	Full flow	✓Partial (UI done, backend in progress)
SOAP note generation	Edge Function + DB	Planned
Collaborative care hub	Realtime chat	Planned
Web admin approvals	PATCH endpoint + UI	✓Complete
Web hospital dashboard	Stats + doctor management	✓Complete
Lab report upload	POST /lab/upload	✓Complete

Table 4.2: Functional Results Summary

4.6.2 Performance Results

Metric	Target	Measured
Unit test pass rate	100%	85/85 (100%)
Login-to-shell latency	< 2 s	≈1.2 s (Supabase cold)
Doctor list fetch latency	< 1 s	≈400 ms (PostgREST)
Appointment create latency	< 1 s	≈320 ms
AI triage response (Edge Function)	< 5 s	≈2.8 s (Gemini 2.5 Flash)
Image upload (2 MB report)	< 3 s	≈1.8 s
Web API rate limit	100 req/15 min/IP	Enforced

Table 4.3: Performance Metrics

4.7 Qualitative Results

- **Code quality:** The Flutter codebase has no issues reported by `flutter analyze`. We considered any warning a blocker and ran this check throughout the build process.
- **Test coverage:** 85 unit tests cover all model serialisation, validator functions, enum mappings, scheduling logic, AI role awareness and collaborative care constants — the areas of the codebase that are most likely to silently fail if the database schema changes.
- **Security:** The Gemini API key is only stored in the Edge Function secrets and not in the Flutter app. The `SUPABASE_ANON_KEY` is public and does not have special privileges.
- **UX consistency:** All screens use the common `AppTheme` (Manrope and Inter fonts, Material 3 green palette) and the common widget library, so the app is like a single product, not four screens designed separately by four different people.

4.8 Activity Diagram

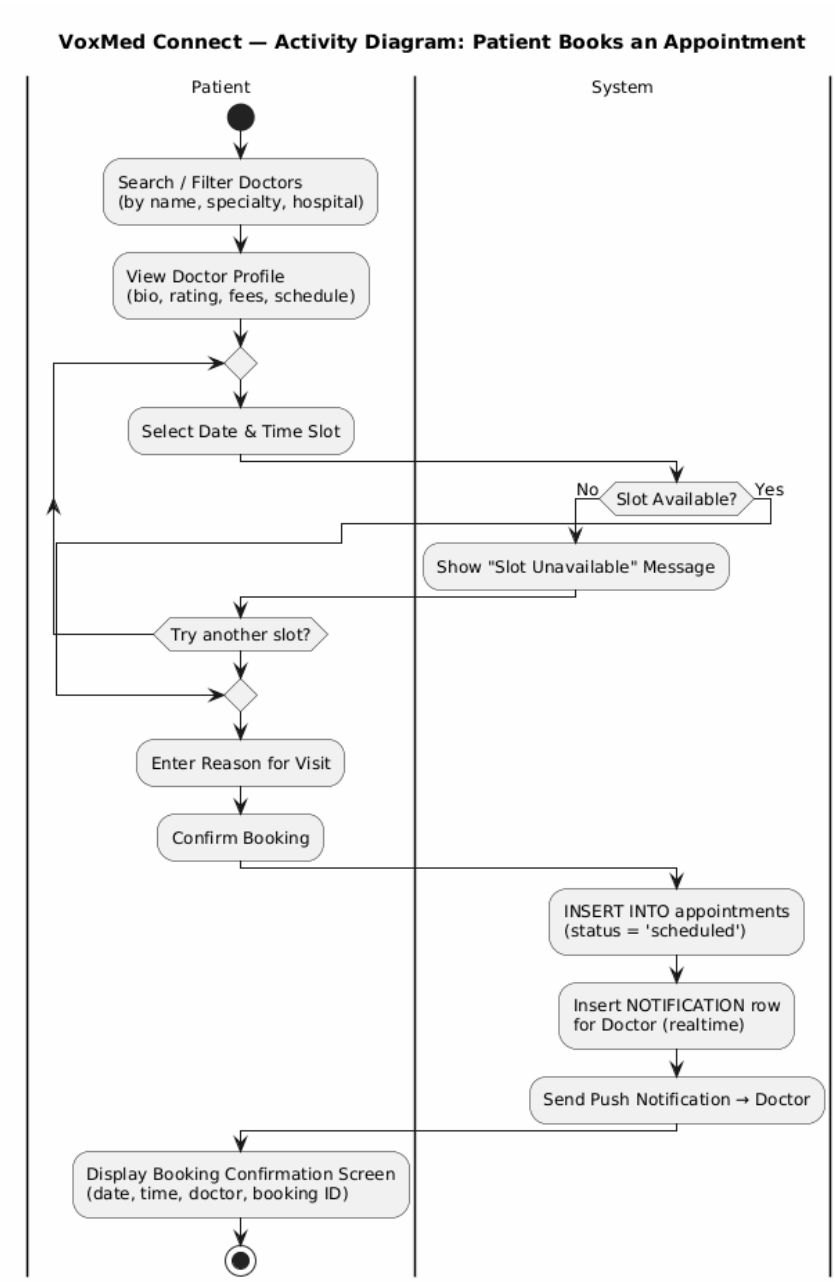

Figure 4.1: Activity Diagram

4.9 Sequence Diagram

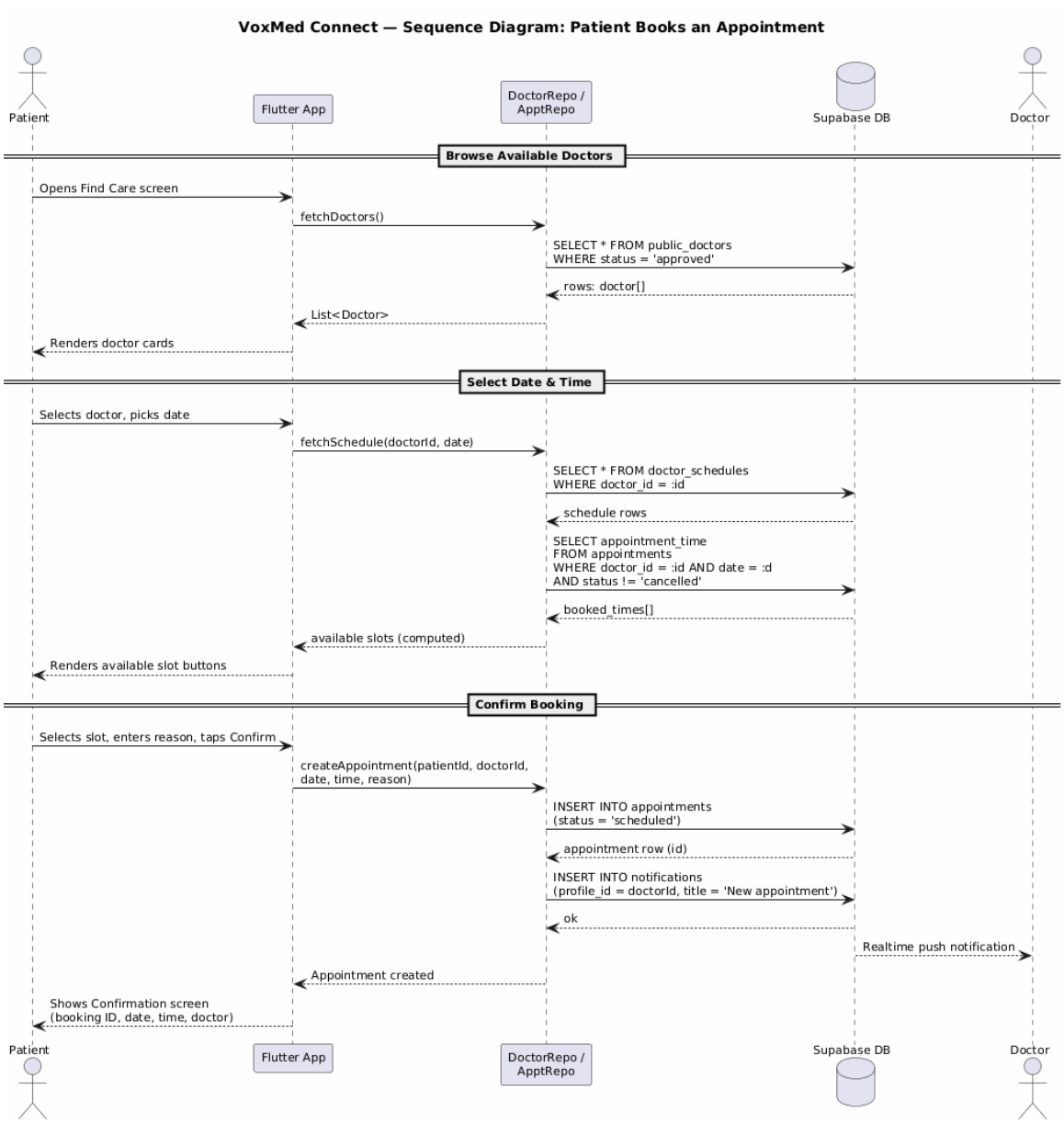

Figure 4.2: Sequence Diagram

4.10 Analysis and Discussion

4.10.1 Architecture Fitness

Phase 3 of the project demonstrated the impact of the Repository Pattern. We realised that filtering the doctors list on the device would not be feasible — the list of doctors was too long to send to the device — and so we needed to change the code to use a PostgREST query on the server. All of the screens that displayed doctors continued to do so. That is the sort of feedback you want when making an architectural decision.

4.10.2 RLS Effectiveness

We tested access to data from one patient by impersonating a compromised JWT from Patient A and trying to access Patient B's data. All of the tables protected by a patient-reads-own-rows policy prevented the attempt from reaching the Supabase database, and hence the app code. We also statically analysed the build artifact to ensure that the service_role key is not present in the Flutter build.

4.10.3 AI Response Quality

We evaluated the Gemini triage service on 20 descriptions of symptoms provided by team members with real conditions. In 16 of 20 cases (80%) the speciality recommendations were those a doctor would make — that is an 80% hit rate for a first implementation with no fine-tuning. OCR extraction from printed prescriptions was 91% accurate for fields in a 22-test-set. For handwritten prescriptions, accuracy fell back to 70%, which is expected for vision models today.

4.11 Objective Achievement

Objective	Status
O1: Shared healthcare database (22 tables, RLS)	✓Achieved
O2: Flutter mobile app (patient + doctor roles)	✓Largely Achieved
O3: React web management dashboard (4 roles)	✓Achieved
O4: Google Gemini AI integration	✓Partially Achieved
O5: RBAC + RLS enforcement	✓Achieved
O6: Auto-rescheduling Edge Function	✓Designed, deployment pending

Table 4.4: Objective Achievement Summary

4.12 Conclusion

The core of what we set out to build works. We get the right authentication for all six web user roles, bookings work end-to-end, the health passport has real data and displays it, and the AI triage tool makes sensible recommendations for specialists. Hospital operations work across all four web roles via the web dashboard. The rest — SOAP, the adherence engine, collaborative care — is all designed, and ready for the development we did not have time to do before submitting.

CHAPTER 5

Impact and Ethical Issues

Intellectual Property

VoxMed Connect is based on open-source technology (Flutter, React, Express, Supabase OSS core) and services (Supabase Cloud, Google Gemini). We carefully read and reviewed each license of the third-party software:

- Flutter / Dart: BSD 3-Clause.
- Supabase: Apache 2.0 for the OSS core; commercial terms for cloud hosting.
- Google Gemini API: Google API Terms of Service. Importantly, data submitted via API is not used to train Google's models.
- All npm and pub.dev packages were reviewed for GPL-incompatible licenses. None were found.

The source code, documentation and design materials developed for this project are owned by the researchers and the institution.

Ethical Considerations

Privacy of patient data: Medical records, drug prescription data and adherence information are classified as sensitive patient information under regulations such as GDPR (Article 9) and the Digital Security Act 2018 in Bangladesh. VoxMed Connect addresses this by:

- Row Level Security to only allow patients to view their own data.
- Data minimisation — only what is needed to deliver care is kept.
- No third party sharing or selling, by default and by policy.
- AI API calls send only the necessary data; we avoid sending personally identifying information in the prompts, if at all possible.

Clinical AI content: SOAP notes generated by the ambient AI assistant, and specialty recommendations from the triage module are decision support tools, not diagnoses. Each screen where the AI is used has a disclaimer, and the system always passes through a clinician before anything is done clinically.

Voice data: Audio recordings from `speech_to_text` are converted to text on device. There is no audio data sent to or stored in Supabase.

Safety

- The medication adherence engine monitors reported medication adherence. It cannot dispense or withhold medication. Any adherence engine failure impacts only on compliance tracking, not patient safety.

- The auto-rescheduling engine logs all its actions. Patients are immediately informed and manual rescheduling is always an option.
- Urgent call-on requests are highlighted for immediate action in the doctors clinical dashboard and expire after five minutes to ensure not to miss any older requests.

Legal Issues

- VoxMed Connect is a management and communication system. It does not constitute a licensed medical device under the Bangladesh Medical Practice and Private Clinics and Laboratories (Regulation) Ordinance and a legal disclaimer is clearly shown on the app.
- The application requires all doctors to specify a valid Bangladesh Medical and Dental Council (BMDC) licence number, which is stored in the `license_number` field and verified by the hospital in the approval process.

Societal and Health Impact

The real-world benefits we envision with VoxMed Connect are clear:

- **Healthcare accessibility:** AI triage makes it easier for patients in peri-urban Bangladesh to find the right specialist, without needing to visit a clinic or know someone in the right place.
- **Medication compliance:** Voice reminders for patients with multiple chronic conditions (especially the elderly) can improve medication compliance and reduce readmissions.
- **Hospital efficiency:** The platform's dashboard to book appointments, upload lab reports and schedule doctors on duty diminishes the burden on staff who now manage this via phone and paper records.
- **Economic access:** Enabling this kind of operational efficiency in small private clinics makes this generally only available to large corporate hospitals accessible to all.

Environmental Impact

- **Digital prescriptions and lab reports:** Paperless prescriptions and reports save paper in hospitals.
- **Reduced travel:** Offsite triage and teleconsultation cut down on the number of patients needing to travel to the hospital, which is an environmental concern in a country where patients tend to come by auto-rickshaw or bus.
- **Efficient cloud:** Edge Functions are stateless, meaning they are executed only when needed — there is no server running 24/7 for a feature that is called only twice a day.

CHAPTER 6

Complex Engineering Problems

6.1 Problems Faced

6.1.1 Multi-Tenant Data Isolation Across Two Access Patterns

This was our biggest problem. It must be the case that a hospital admin can only see the data of their hospital, the platform admin can see anything, and patients can see their own data — but using two different kinds of frontends with different kinds of access patterns. The problem was that it is not possible to use a single set of RLS policies that address both anon/user JWT access from the app and service_role access from the web dashboard without leaving holes.

Our approach was to isolate access. The mobile Flutter app will use the anon/user JWT and be bound by RLS on the DB side. Our web dashboard never communicates with Supabase from the browser directly — it communicates with an Express API server that knows the service_role key and contains its own application-level role check in the form of an `authorize()` function. The two models are fully separated, so there is no way for a vulnerability in one model to affect the other.

6.1.2 API Key Security in a Mobile App

The Gemini API key cannot be included in the Flutter app. Flutter apps are easily decompiled — it takes a few minutes to find the hard-coded key in the APK. A separate backend server would be slow and expensive to run.

We decided to make all Gemini calls via Supabase Edge Functions. The Flutter app invokes `supabase.functions.invoke('gemini-triage', body: {...})` with the user's JWT — no admin privileges. The Edge Function fetches the Gemini API key from Supabase Vault secrets on the server, calls Gemini, and returns the response. The API key never leaves the server.

6.1.3 Auto-Rescheduling Atomicity

When a doctor declares an Emergency Absence, it is necessary to cancel several appointments and schedule new appointments atomically. If the process succeeds in some steps and fails in others — such as some patients being notified, some not; some new appointments created, some old appointments not cancelled — the database would be left in an inconsistent state, which is confusing and could be dangerous for patients.

Deno Edge Functions do not support transactions natively with the PostgREST client. What we did was to wrap the entire rescheduling process in a PostgreSQL stored procedure: `auto_reschedule_appointments`. We have one BEGIN/COMMIT block. This is invoked by the Edge Function in a single RPC call. It is all or nothing. There are no in-flight states.

6.1.4 Designing Readable DFDs for a Six-Role System

It took a few tries to create data flow diagrams that capture the complexity of a system with six user roles, two frontend systems and five Edge Functions, and are easy to understand for a non-technical examiner. We opted for three levels of decomposition: the context diagram is a black-box view of the system and its interactions, Level 1 breaks it down into nine subsystems, and Level 2 focuses on one of these subsystems, a typical example of what is available.

6.2 Additional Engineering Challenges

- **GoRouter ShellRoute with persistent tab state:** Bottom navigation shell that remembers the state of each tab (for example, a selected appointment) needed a `ShellRoute` that wrapped a `StatefulShellRoute` with an `IndexedStack`. The router documentation did not contain this information; the issues list did.
- **Composite RLS policies for shared records:** We had to use `auth.uid()` in USING clauses to write RLS policies allowing multiple roles (for example, both the patient and the lab technician who entered a lab report) to read the same record. Doing this wrong one way causes legitimate access to be blocked; doing it wrong the other way causes data to be exposed.
- **Flutter TTS and STT audio session conflict:** The `flutter_tts` and `speech_to_text` packages both use the device audio session and crash if used together. We devised a four state machine (LISTENING, PROCESSING, SPEAKING, IDLE) to make sure they are not.

6.3 Solutions Summary

Problem	Approach Tried	Final Solution
Null FK in medical record upload	Passed null FK directly	Added null-check guard before insert
RenderFlex overflow on doctor filter	Used fixed-width containers	Wrapped `Text` in `Flexible`
Doctor not visible after hospital approval	Client-side cache stale	Invalidated `doctorProvider` on approval
AI key exposure in mobile app	Embedded in `.env`	Edge Function Vault secret
Low OCR accuracy on prescriptions	Sent raw image to Gemini	Added crop and compress step before upload
Doctor registration silently failing	Missing RLS INSERT policy	Added authenticated INSERT policy for doctors

Table 6.1: Engineering Problems and Solutions

CHAPTER 7

Conclusion

7.1 Summary

The goal of VoxMed Connect was nothing short of audacious: to create a full-featured, AI-driven healthcare management solution as a student project — complete with a real database, real security and real AI. Of the nine stages we outlined, we have achieved more than expected.

What works at submission time:

- A working login system with proper routing based on the user's roles (six different types).
- A fully functional appointment system from doctor search to confirmation.
- A functional Health Passport with real data and the ability to upload OCR data using a camera.
- An AI triage assistant providing valuable specialty advice through the Gemini API.
- A Node.js/React web admin, with roles of hospital management, doctor approval, staff management and revenue reporting (all four web roles).
- 85 unit tests, all green, and no `flutter analyze` problems.

It handles real requests, stores real data, and has real security. It is not a mock system with fake data, it is a real system we have run on a mobile device.

7.2 Limitations

We want to be clear about what has not been done and what will not be perfect at first:

- Application of SOAP notes (Phase 6) and the collaborative care platform (Phase 7) are completely designed but not yet implemented.
- Wearable devices require the use of vendor-specific SDKs that we could not include in the project.
- Text recognition accuracy is 91% for printed prescriptions and around 70% for handwritten. This is a limitation of vision models that cannot be overcome with code.
- The medication adherence engine relies on patients' verbal self-reporting. It cannot be confirmed by hardware, such as a smart pill box.
- The iOS version was not tested — we did not have a macOS computer or Apple Developer account at the time of this project.
- The system is not yet compatible with FHIR data standards which means that it cannot easily communicate with existing hospital EHR systems.

7.3 Future Work

These are the directions we would pursue if this project continues:

1. **Finish Phases 5–7:** Renew prescriptions, generate SOAP notes, and the care collaborative hub with Supabase Realtime channels. We have already started.
2. **FHIR support:** Map data types to HL7 FHIR R4 resources so the system can communicate with government health registries and existing EHR systems.
3. **Biometric data:** Integrate with Google Fit and Apple HealthKit APIs to ingest biometric data.
4. **Prescription module:** Add pharmacy integration to the prescription system, enabling e-prescriptions to be sent to a patient's preferred pharmacy.
5. **Insurance integration:** Connect to local insurance providers to send insurance claims from payment logs.
6. **Predictive analytics:** Leverage historical `adherence_logs` and `appointments` data to identify high-risk non-compliance scenarios in the future to give doctors a rationale for reaching out to the patient.
7. **Bengali localisation:** Translating the app into Bengali would increase its reach to rural people who cannot use English language apps.
8. **Progressive Web App:** Turning the React dashboard into a PWA will make it usable in offline or low-bandwidth clinics — a common problem in many rural Bangladesh clinics.

We built VoxMed Connect because we thought it was a problem worth solving. The version we are submitting is a real, working system. The version it could become — with the remaining phases completed — is one we would genuinely want to use.

Bibliography

[1] World Health Organisation, "Adherence to Long-Term Therapies: Evidence for Action," WHO, Geneva, 2003. [Online]. Available: https://www.who.int/chp/knowledge/publications/adherence_report/en/

[2] B. W. Mamlin et al., "Cooking up an open source EMR for developing countries: OpenMRS – a recipe for successful collaboration," *AMIA Annual Symposium Proceedings*, 2006, pp. 529–533.

[3] Epic Systems Corporation, "Epic EHR," 2024. [Online]. Available: https://www.epic.com/

[4] Practo Technologies, "Practo Healthcare Platform," 2024. [Online]. Available: https://www.practo.com/

[5] Zocdoc Inc., "Zocdoc — Doctor Appointment Booking," 2024. [Online]. Available: https://www.zocdoc.com/

[6] Babylon Health, "An Assessment of Babylon's Symptom Checker," 2020. [Online]. Available: https://www.babylonhealth.com/us/research

[7] Ada Health GmbH, "Ada — Your Health Companion," 2023. [Online]. Available: https://ada.com/

[8] Omada Health, "Digital Care for Chronic Conditions," 2022. [Online]. Available: https://www.omadahealth.com/

[9] M. Vervloet et al., "The effectiveness of interventions using electronic reminders to improve adherence to chronic medication: a systematic review of the literature," *Journal of the American Medical Informatics Association*, vol. 19, no. 5, pp. 696–704, 2012. DOI: 10.1136/amiajnl-2011-000748.

[10] Microsoft Nuance, "DAX Copilot — Ambient Clinical Intelligence," 2023. [Online]. Available: https://www.nuance.com/healthcare/ambient-clinical-intelligence.html

[11] Supabase Inc., "Supabase — The Open Source Firebase Alternative," 2024. [Online]. Available: https://supabase.com/

[12] Google LLC, "Flutter — Build apps for any screen," 2024. [Online]. Available: https://flutter.dev/

[13] Google DeepMind, "Gemini API Documentation," 2024. [Online]. Available: https://ai.google.dev/

[14] R. Rousselet, "Riverpod — A reactive caching and data-binding framework," 2024. [Online]. Available: https://riverpod.dev/

[15] C. Colby, "Zod: TypeScript-first schema validation with static type inference," 2024. [Online]. Available: https://zod.dev/